



CI/CD for ML: From Notebook → Production

Automate build, test, scan, and deploy processes to enable reproducible releases of **code + model + data** with faster iterations protected by **safety gates**.

Why CI/CD for ML Is Different

Traditional CI/CD pipelines aren't designed for the unique challenges of machine learning systems:

Complex Artifacts

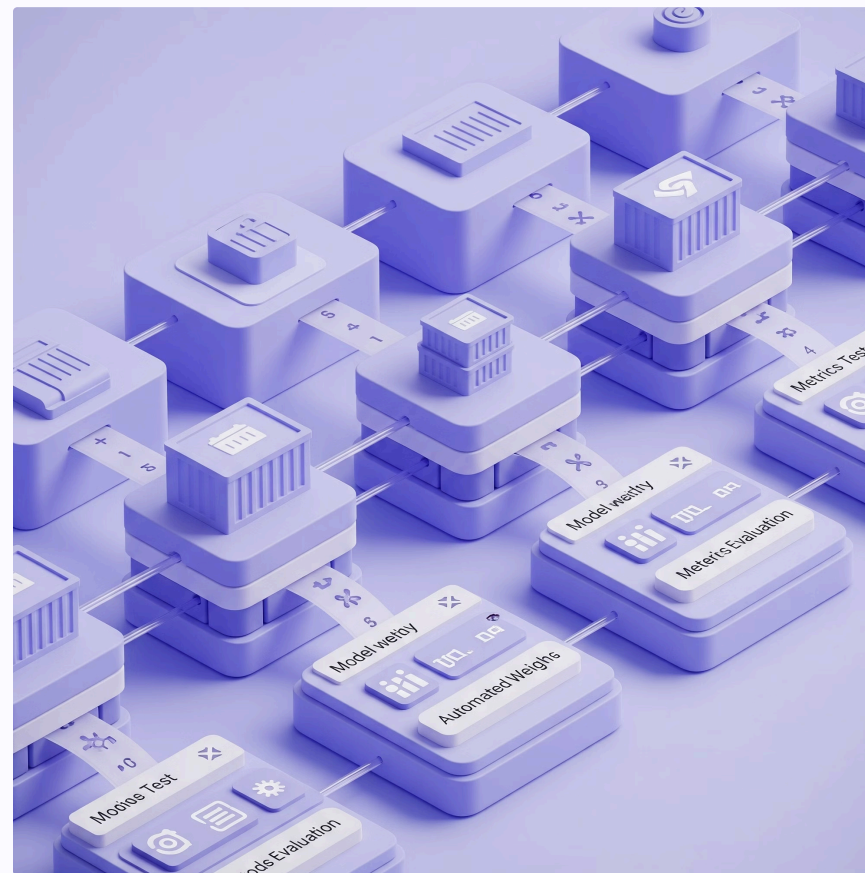
Container images + model weights + data/feature versions all must be versioned together

Non-determinism

Requires metrics & drift checks integrated into pipeline steps

Multi-dimensional Promotion

Must consider accuracy + latency + cost budgets together



Target Pipeline (High-Level)



CI

Lint → unit tests → model tests → build image → scan → push



Package

Helm/Kustomize manifest → sign & store



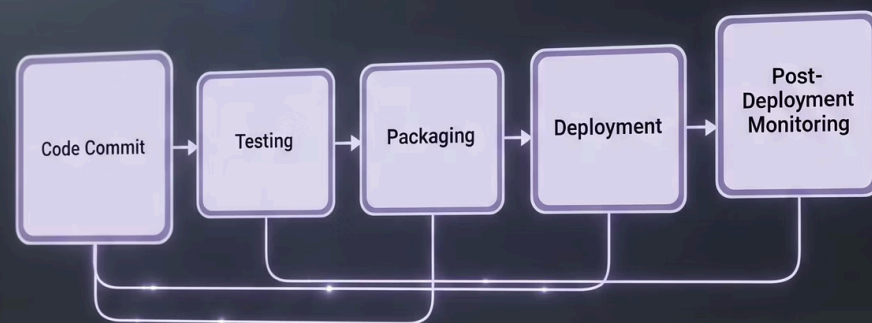
CD

Deploy to dev → staging → prod via GitOps



Post-deploy

Smoke tests → canary analysis → auto-promote/rollback



Branching & Releases

Branch Strategy

main

Production candidate branch

develop

Staging environment branch

feature/*

PR checks and validation

Tagging Artifacts

Use semantic versioning (v1.3.0) consistently across:

- Container images
- Helm charts
- Registry references
- Model registry versions



Artifacts & Registries

Container

GHCR/ECR/GAR

ghcr.io/org/app:1.3.0

Model

MLflow Model Registry

llm:v12 → PROD

Manifests

Helm chart

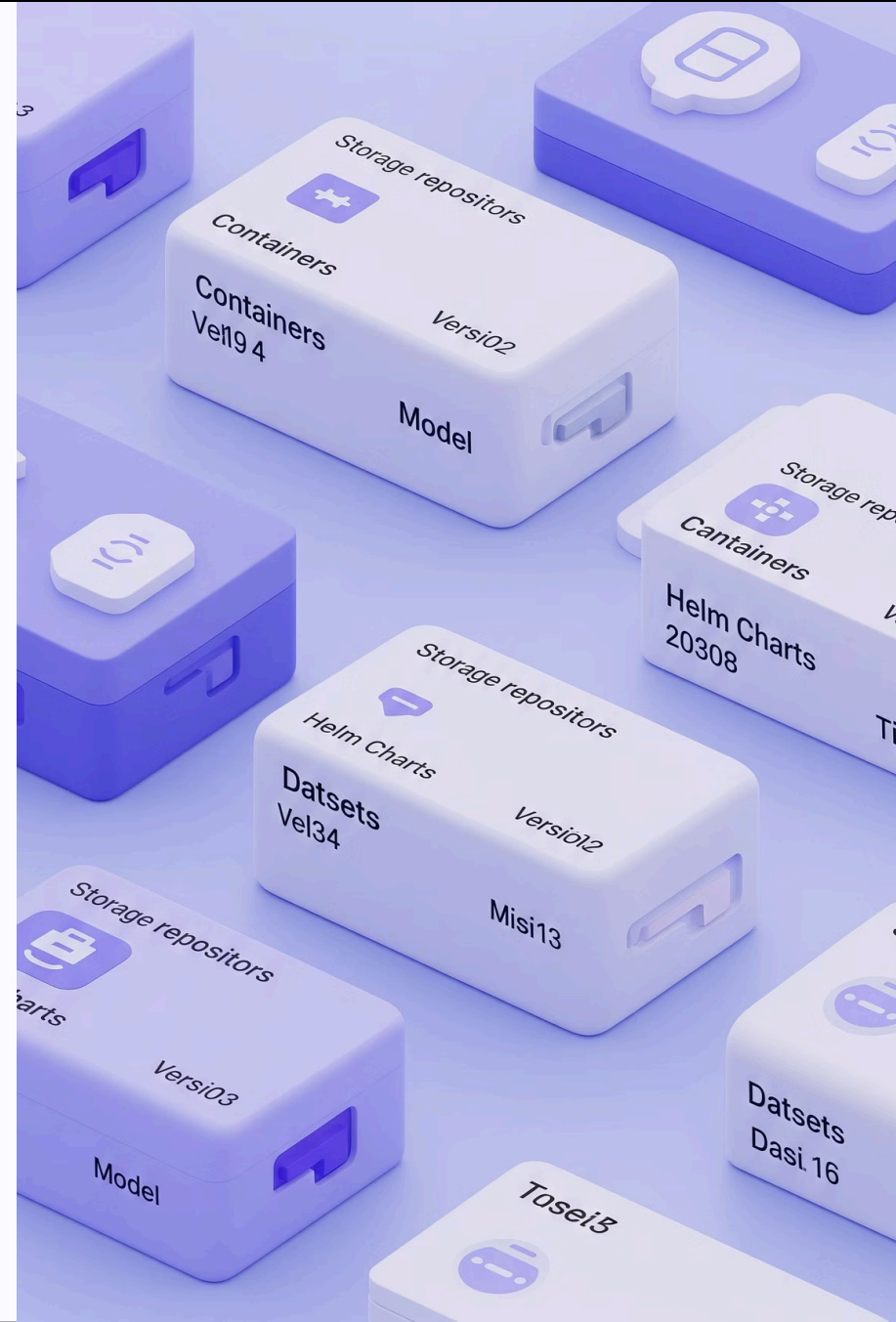
charts/app-1.3.0.tgz

Data

DVC/LakeFS version

dataset@sha256:...

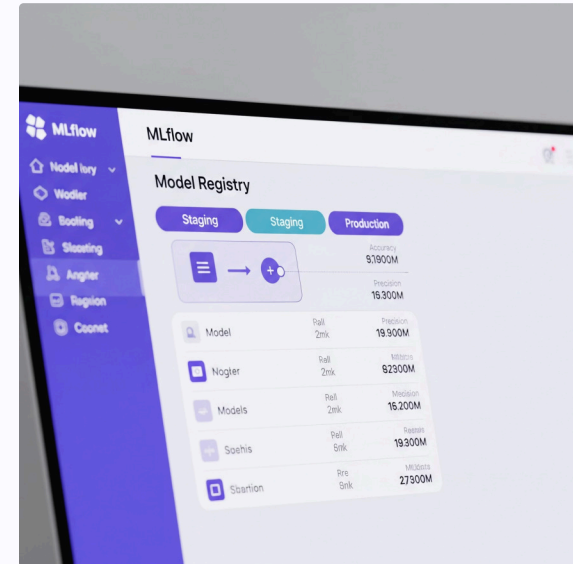
Each artifact type requires its own specialized registry with versioning capabilities and access controls.



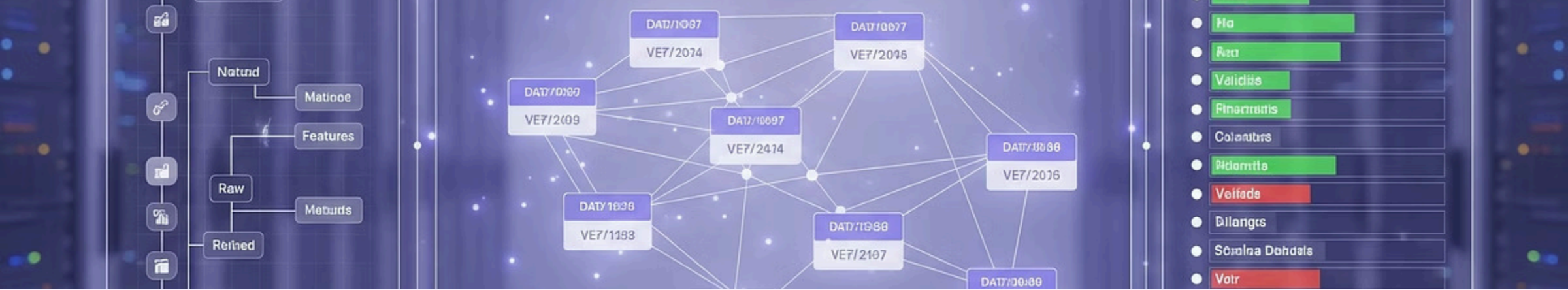
Model Registry Integration (MLflow)

Key Integration Points

- Register new run with comprehensive metrics
- Implement promotion gates:
 - Require $\geq$ **target F1 score**
 - Require $\leq$ **p95 latency** before promotion
- Transition stages via CI jobs:
 - **STAGING** $\rightarrow$ **PRODUCTION**



The MLflow Model Registry serves as the central hub for managing model lifecycle states and ensuring only qualified models reach production.



Data & Feature Versioning



Dataset Pinning

DVC/LakeFS creates immutable snapshots of datasets for reproducibility



Feature Materialization

Feast handles offline → online materialization with hash verification



Schema Validation

Pipeline checks refuse deployment if **feature schema drift** is detected

Data versioning is equally important as code versioning - changes in data can impact model performance even with identical code.

Tests You Need

Unit Tests

Preprocessing functions, utility methods, data transformers

Model Tests

Deterministic smoke tests, baseline metric comparisons

API Tests

Contract validation (OpenAPI), latency smoke tests

Load Tests

p95/p99 latency thresholds, error rate validation



ML systems require testing both the code quality *and* the model quality, making the test suite more complex than traditional applications.

- ❏ Run high-cost load tests only in staging environments to minimize resource usage.

Performance & Cost Budgets

Latency Budget

Block release if **p95 latency** > **300ms** in staging canary tests

Cost Efficiency

Block if **\$ per 1k requests** increased > 15% from baseline

Resource Utilization

Block if **GPU utilization** < **30%** under load testing (inefficient resource usage)

Performance budgets prevent regressions and help maintain cost-effective deployments. Each budget should be tuned to your specific application requirements.



Security & Compliance Gates

Security Scans

- **SAST** (Static Application Security Testing)
- **Dependency scanning** for vulnerable packages
- **Container scanning** (Trivy/Grype)
- **SBOM** generation (Software Bill of Materials)

Governance

- **Policy-as-code** (OPA/Conftest)
 - Required labels & metadata
 - Resource limits enforcement
 - No `:latest` tags allowed
- **Sigstore cosign** for artifact signing



Security gates prevent vulnerable or non-compliant ML systems from reaching production environments, protecting both your infrastructure and your data.

Example GitHub Actions (CI)

```
name: ci-build-test
on: [pull_request, push]
jobs:
  build-test:
    runs-on: ubuntu-latest
    permissions: {contents: read, packages: write, id-token: write}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.10'}
      - run: pip install -r requirements.txt && pip install pytest mlflow
      - name: Unit & model tests
        run: pytest -q
      - name: Build image
        run: docker build -t ghcr.io/org/nlp:${{ github.sha }} .
      - name: SBOM & scan
        run: |
          curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh | sh -s -- -b .
          ./syft packages ghcr.io/org/nlp:${{ github.sha }} -o spdx-json > sbom.json
          docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest \
            image --exit-code 1 ghcr.io/org/nlp:${{ github.sha }}
      - name: Login & push
        uses: docker/login-action@v3
        with: {registry: ghcr.io, username: ${{ github.actor }}, password: ${{ secrets.GITHUB_TOKEN }}}
      - run: docker push ghcr.io/org/nlp:${{ github.sha }}
```

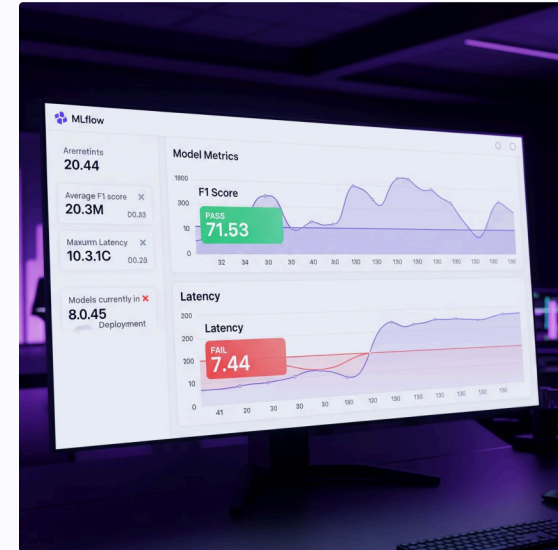
This workflow handles testing, building, scanning, and publishing container images. Additional steps for model registry updates would follow similar patterns.

MLflow Gate (CI Step)

```
python ci/mlflow_gate.py \  
  --run-id $RUN_ID \  
  --require-f1 0.85 \  
  --max-latency-ms 300 \  
  --promote-if-pass STAGING
```

This script:

- Retrieves metrics from a specific MLflow run
- Validates against threshold requirements
- Fails the pipeline if targets are missed
- Automatically promotes to **STAGING** if all checks pass



Gates like this ensure that only models meeting both accuracy and performance requirements advance to the next stage.

 Custom gates can be created for any metrics tracked in your ML platform.



GitOps CD (Argo CD)

GitOps Workflow

- Application configs live in dedicated **config repository**
- CI pipeline opens PR to update image tag/Helm chart
- Argo CD continuously reconciles clusters to match repository state
- Rollback = simple git revert operation

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: {name: nlp-staging}
spec:
  source:
    repoURL: https://github.com/org/infra-config
    path: charts/nlp
    targetRevision: main
  helm: {parameters: [{name: image.tag, value: "${GIT_SHA}"}]}
  destination:
    {server: https://kubernetes.default.svc, namespace: nlp}
  syncPolicy: {automated: {prune: true, selfHeal: true}}
```

Direct CD (Actions + kubectl)

Alternative Approach

GitHuds

```
#!/bin/bash
set -e

# Deploy directly to the cluster
kubectl apply -f deployment.yaml

# Verify the deployment
kubectl get pods
```

```
name: cd-staging
on: workflow_dispatch
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: azure/setup-kubectl@v4
      - uses: azure/k8s-set-context@v4
        with:
          method: kubeconfig,
          kubeconfig: ${{ secrets.KUBE_CONFIG }}
      - run: |
          helm upgrade --install nlp charts/nlp \
            --namespace nlp --create-namespace \
            --set image.tag=${{ github.sha }}
```

While GitOps is preferred, direct CD via GitHub Actions offers:

- Simpler initial setup
- Familiar workflow for teams new to GitOps
- Good starting point for migration to GitOps

⊗ Direct CD lacks the full audit trail and reconciliation capabilities of GitOps approaches.

Canary / Blue-Green Strategy

Canary Deployment

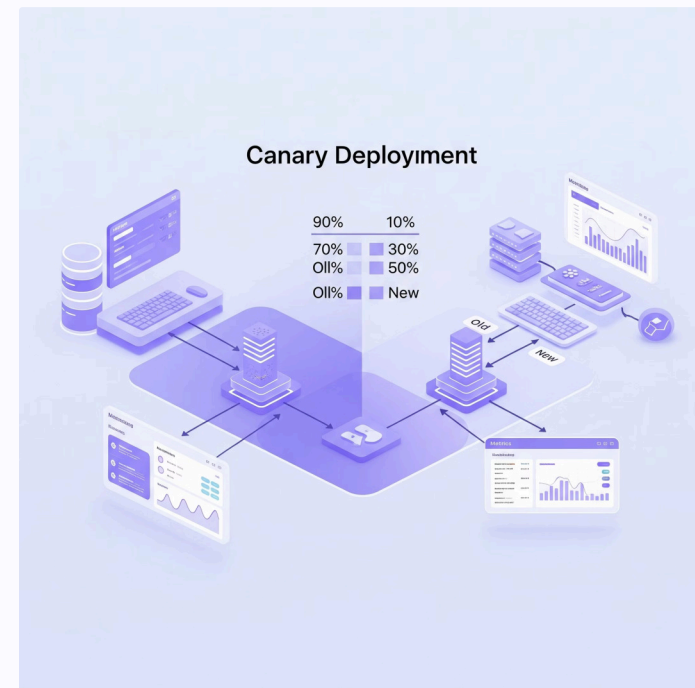
5% → 25% → 50% traffic; abort if error/latency exceeds SLO

Blue-Green Deployment

Stand up GREEN environment; switch Service selector atomically

Istio VirtualService Example

```
# Istio VirtualService (weight split)
spec:
  http:
    - route:
        - destination: {host: nlp-svc, subset: v1, weight: 95}
        - destination: {host: nlp-svc, subset: v2, weight: 5}
```



Progressive deployment strategies reduce risk by exposing new ML models to limited traffic first, enabling validation with real users before full rollout.



Auto-Rollback & Promotion

Monitor

Canary analysis job continuously queries Prometheus for **p95 latency** and **error rate** metrics

Success Path

If metrics pass thresholds → promote to 100% traffic + tag model as **PROD** in model registry

Evaluate

Compare metrics against SLO thresholds for a defined evaluation period (e.g., N minutes)

Failure Path

If metrics fail thresholds → Argo Rollouts / Actions automatically revert to previous stable version

Secrets & Identity

Workload Identity

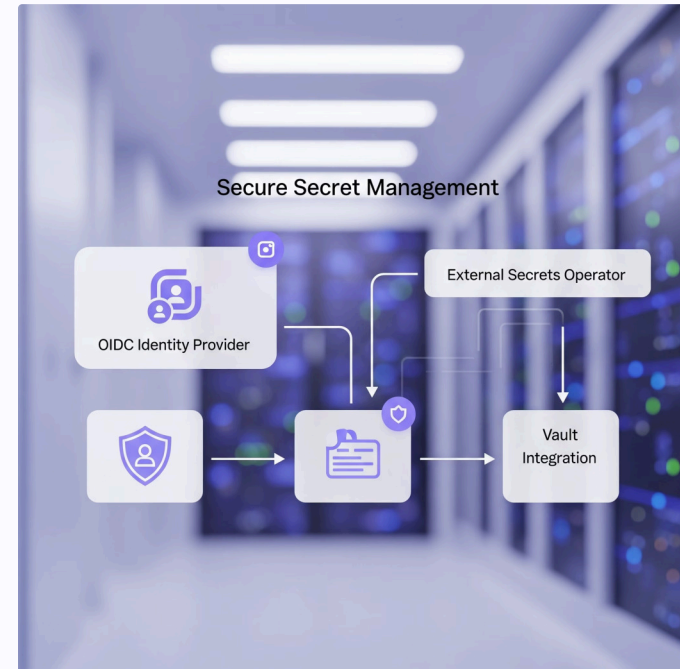
Prefer **OIDC workload identity** for cloud access (eliminates static keys)

Secret Management

Store secrets in **External Secrets** backed by cloud KMS or HashiCorp Vault

Policy Enforcement

Disallow secrets in manifests; enforce via OPA policy checks



Modern secret management reduces attack surface by eliminating long-lived credentials and centralizing secret management with proper access controls.

Environment Config Management

Helm Values Approach

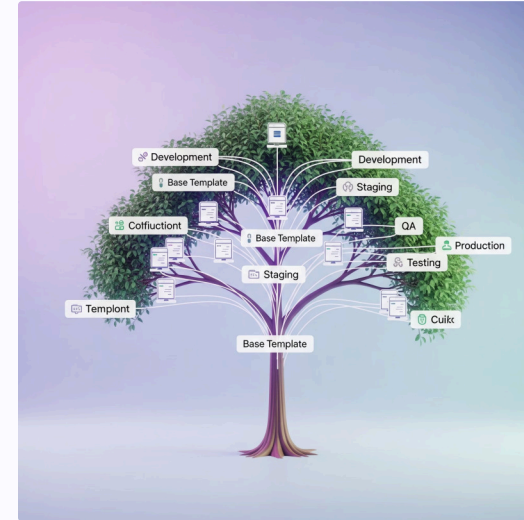
Use environment-specific values files:

- values-dev.yaml
- values-staging.yaml
- values-prod.yaml

Kustomize Approach

Use environment-specific overlays:

- overlays/dev/kustomization.yaml
- overlays/staging/kustomization.yaml
- overlays/prod/kustomization.yaml



Key environment differences to configure:

- Replica counts and autoscaling thresholds
- Resource quotas and limits
- External service URLs and endpoints
- Monitoring intensity and alerting thresholds

Maintain one chart/base and promote using values/overlays only

Observability Hooks in CI/CD



Health Probes

Enforce readiness/liveness probe configurations in all deployments



Smoke Testing

Run post-deploy jobs that hit `/healthz` & `/predict` endpoints with sample data



Release Markers

Export deployment events to Grafana to correlate metrics with specific releases

Integrating observability directly into the CI/CD pipeline provides immediate feedback on deployment success and helps quickly identify the source of regressions.



FinOps in the Pipeline

Cost-Aware Deployments

- CI job estimates **cost per 1k requests** based on:
 - Model size
 - Hardware requirements
 - Average latency
- Pipeline blocks deployment if cost delta exceeds threshold
- Proper resource tagging enables detailed cost attribution:
 - team
 - environment
 - costCenter



Integrating cost awareness into ML pipelines prevents unexpected budget overruns and encourages optimization of both model performance and resource efficiency.

Minimal Starter Checklist

1 CI Basics

Tests, image build, SBOM + vulnerability scanning, image push

2 Essential Gates

MLflow metric checks, policy-as-code validation, performance budget verification

3 CD Foundation

GitOps application per environment; canary/blue-green capability & auto-rollback

4 Security Fundamentals

OIDC workload identity + cloud KMS/Vault for secrets

5 Basic Observability

Post-deployment smoke tests + release markers in monitoring

Summary

Unified Artifacts

Treat **code + model + data** as first-class release artifacts with coordinated versioning

Comprehensive Gates

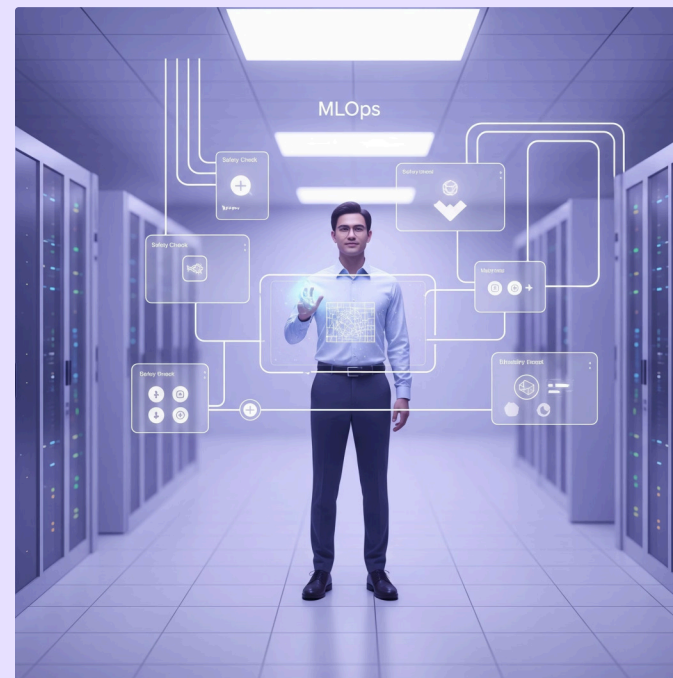
Add **security, performance, and cost gates** to your CI/CD pipeline

GitOps Deployment

Use **GitOps** for reliable, auditable environment promotion

Safe Velocity

Combine canary deployments with auto-rollback for safe, continuous delivery



Effective ML CI/CD enables teams to move quickly with confidence, knowing that automated guardrails protect against quality, performance, and cost regressions.

Discussion Prompt

For your Capstone...

Will you start with **simple CI + manual prod deploy** or
go **full GitOps with canary & auto-rollback**?

What's your gating metric?

